

蒙特卡洛流体仿真实验报告

2022 涡量法与 2024 速度法的实现与对比

姓名：侍子译 学号：PB23000284

August 1, 2026

Abstract

本实验实现了两种基于 Monte Carlo 的不可压缩流体仿真方法，并进行了系统对比：2022 年 Rioux-Lavoie et al. 提出的**涡量法** [1]（涡量输运 + Biot-Savart 重建速度 + 流函数 WoS 处理边界），以及 2024 年 Sugimoto et al. 提出的**速度法** [2]（速度算子分裂 + Walk-on-Boundary 投影维持不可压缩性）。两种方法的根本区别在于流体状态的表示：涡量法以标量涡量场 ω 为状态变量，通过 Biot-Savart 积分重建速度，天然满足 $\nabla \cdot \mathbf{u} = 0$ ；速度法直接以速度场 $\mathbf{u}$ 为状态，通过投影步骤显式维持不可压缩性。实验在 Taylor-Green 涡流场上对两种方法进行了对比，考察了动能演化、涡量守恒和计算开销。结果表明：涡量法计算代价高（每步约为速度法的 18 倍），但物理量（环量、涡量）守恒性更好且不存在压力投影的噪声反馈问题；速度法计算高效，但依赖投影步骤且数值耗散较大。两种方法各有适用场景，涡量法更适合强调物理守恒的研究性仿真，速度法更适合追求效率的动画级应用。

引言

传统的计算流体力学方法通常依赖网格离散化，在复杂几何边界上面临网格生成困难。近年来，基于随机游走的 Monte Carlo PDE 求解方法——特别是 Walk-on-Spheres (WoS)——被引入流体仿真领域，提供了完全无网格的求解途径。

2022 年，Rioux-Lavoie、Sugimoto 等人提出了首个 Monte Carlo 流体仿真框架 [1]。该方法采用**涡量-速度**表示：以涡量 ω 为状态变量，通过 Biot-Savart 定律由涡量重建速度，边界条件通过流函数 ψ 与 WoS 算法处理。由于 Biot-Savart 重建的速度场天然无散，该方法**无需压力投影**，从根本上避免了不可压缩性约束的数值实现难题。

2024 年，Sugimoto、Batty 和 Hachisuka 提出**基于速度**的 Monte Carlo 方法 [2]。该方法直接以速度场 $\mathbf{u}$ 为状态变量，采用经典算子分裂框架（平流 $\rightarrow$ 外力 $\rightarrow$ 扩散 $\rightarrow$ 投影），其中投影步骤用 Walk-on-Boundary (WoB) 边界积分直接估计压力梯度 ∇p 。由于速度法直接使用速度场，可以方便地融入浮力、PIC/FLIP 等传统速度法技术，但需要显式维持 $\nabla \cdot \mathbf{u} = 0$ 。

本实验对比实现了这两种算法：

- **2022 涡量法**：参考合作者的实现 (2022_Simulation/), 包括 2D 涡量输运 + Biot-Savart、流函数 WoS 边界、3D 涡拉伸等完整管线；
- **2024 速度法**：本实验实现 (fluid_sim_wob.py), 速度算子分裂 + WoB 特解投影。

两者均使用 Numba JIT 加速，在 Taylor-Green 涡流场上从相同初始条件出发进行对比，考察动能演化、涡量守恒和计算效率。

算法原理

2022 涡量法

涡量输运方程

2022 年方法的理论基础是二维不可压缩流的**涡量输运方程**。对于无粘外流，涡量 $\omega = \partial v / \partial x - \partial u / \partial y$ 沿流线输运：

$$\frac{D\omega}{Dt} = \frac{\partial \omega}{\partial t} + \mathbf{u} \cdot \nabla \omega = 0. \quad (1)$$

采用半拉格朗日后向追踪离散：

$$\omega(\mathbf{x}, t) = \omega(\mathbf{x} - \Delta t \mathbf{u}(\mathbf{x}, t), t - \Delta t). \quad (2)$$

Biot-Savart 重建速度

从涡量场重建速度场的关键是 Biot-Savart 定律。在 2D 中，速度与涡量的关系为

$$\mathbf{u}(\mathbf{x}) = \int_{\mathbb{R}^2} \mathbf{B}(\mathbf{x}, \mathbf{y}) \omega(\mathbf{y}) d\mathbf{y}, \quad (3)$$

其中 Biot-Savart 核为

$$\mathbf{B}(\mathbf{x}, \mathbf{y}) = \frac{1}{2\pi} \frac{(y_y - x_y, -(y_x - x_x))}{|\mathbf{x} - \mathbf{y}|^2}. \quad (4)$$

该积分通过 Monte Carlo 估计：在计算域内均匀采样点 $\mathbf{y}_i$ ，估计速度

$$\mathbf{u}(\mathbf{x}) \approx \frac{1}{N} \sum_{i=1}^N \frac{\mathbf{B}(\mathbf{x}, \mathbf{y}_i)}{p(\mathbf{y}_i)} \omega(\mathbf{y}_i), \quad (5)$$

其中 p 为采样概率密度。**关键性质**：Biot-Savart 重建的速度场自动满足 $\nabla \cdot \mathbf{u} = 0$ ，因此涡量法**无需任何投影步骤**。

边界处理：流函数 + WoS

对于存在固体的边界，2022 年论文引入**流函数** ψ 。流函数满足

$$\nabla^2 \psi = -\omega, \quad \mathbf{u} = \nabla \times \psi = \left(\frac{\partial \psi}{\partial y}, -\frac{\partial \psi}{\partial x} \right). \quad (6)$$

流函数 Poisson 方程 $\nabla^2 \psi = -\omega$ 通过 Walk-on-Spheres (WoS) 算法求解。WoS 利用 Brown 运动的均值性质：从点 x_k 出发，到边界的距离为 d_k ，随机跳至球面 $x_{k+1} = x_k + d_k(\cos \theta, \sin \theta)$ 。对 Poisson 方程 $\nabla^2 u = f$ ，Feynman-Kac 公式给出

$$u(x) = \mathbb{E}[g(B_\tau)] - \mathbb{E} \left[\int_0^\tau f(B_s) ds \right], \quad (7)$$

其 WoS 离散形式为源项按 $\frac{d_k^2}{4} f(x_k)$ 累积，直至到达边界 ($d_k < \varepsilon$)。

2024 速度法

算子分裂

2024 年方法直接以速度场为状态，采用经典算子分裂框架（Stam 1999）求解不可压缩 Navier-Stokes 方程：

$$\frac{\partial \mathbf{u}}{\partial t} = -(\mathbf{u} \cdot \nabla) \mathbf{u} - \nabla p + \nu \nabla^2 \mathbf{u} + \mathbf{f}, \quad \nabla \cdot \mathbf{u} = 0. \quad (8)$$

每个时间步分解为四个子步骤：

- (1) **平流**：半拉格朗日后向追踪， $\mathbf{u}^*(\mathbf{x}) = \mathbf{u}^n(\mathbf{x} - \mathbf{u}^n \Delta t)$ ；
- (2) **外力**： $\mathbf{u}^{**} = \mathbf{u}^* + \Delta t \mathbf{f}$ ；
- (3) **扩散**：WoS/Feynman-Kac 求解 $\partial \mathbf{u} / \partial t = \nu \nabla^2 \mathbf{u}$ ；
- (4) **投影**： $\nabla^2 p = \nabla \cdot \mathbf{u}$ ， $\mathbf{u}^{n+1} = \mathbf{u} - \nabla p$ ，维持 $\nabla \cdot \mathbf{u} = 0$ 。

WoB 投影

投影步骤需要求解压力 Poisson 方程 $\nabla^2 p = \nabla \cdot \mathbf{u}$ 。2024 年论文提出用 Walk-on-Boundary (WoB) 边界积分直接估计 ∇p ，而非求解标量 p 。

由于完整 WoB 多弹跳路径积分需要 GPU 加速，本实验采用**特解 + 边界修正**的简化实现。将解分解为自由空间特解 p_p 与调和修正 p_h ：

$$p(\mathbf{x}) = p_p(\mathbf{x}) + p_h(\mathbf{x}), \quad (9)$$

其中

$$p_p(\mathbf{x}) = \int_{\Omega} \frac{1}{2\pi} \ln |\mathbf{x} - \mathbf{y}| f(\mathbf{y}) d\mathbf{y}, \quad p_h(\mathbf{x}) = \mathbb{E}[-p_p(\mathbf{y}_{\partial\Omega})]. \quad (10)$$

p_h 中的期望通过 WoB 射线采样：从 $\mathbf{x}$ 出发沿均匀随机方向发射射线至矩形边界， $\mathbf{y}_{\partial\Omega}$ 为首次撞点。对凸域而言该采样等价于调和测度，因而是正确的。得到 p 后，用中心差分计算 ∇p 修正速度。

两种算法的本质区别

Table 1: 2022 涡量法与 2024 速度法的对比

特征	2022 涡量法	2024 速度法
状态变量	涡量 ω	速度 $\mathbf{u}$
速度重建	Biot-Savart 积分	投影步骤
不可压缩性	天然满足	投影显式维持
压力求解	流函数 ψ (WoS)	∇p (WoB)
边界处理	流函数 + WoS	速度场 + WoB
可融入技术	涡量类（涡拉伸等）	速度类（浮力、PIC/FLIP）
主要代价	Biot-Savart 计算昂贵	投影噪声 / 数值耗散

两种方法的根本差异源于流体状态的表示。涡量法以标量场 ω 表示流体，速度由积分重建，自动满足不可压缩性但计算昂贵；速度法以矢量场 $\mathbf{u}$ 表示，通过投影维持不可压缩性，计算高效但需要处理投影噪声。

代码实现

2022 涡量法实现

2022 涡量法的实现位于 2022_Simulation/, 包含四个递进阶段:

- `mc_fluid_step1_py` (2D 无粘): 涡量输运 + 均匀采样 Biot-Savart + 半拉格朗日平流 + 网格缓存;
- `mc_fluid_step2_py` (2D 边界): 流函数 + WoS 求解 $\nabla^2\psi = -\omega$ + 对偶采样, 支持自由滑移边界;
- `mc_fluid_step3_py` (3D): 3D Biot-Savart + Feynman-Kac 粘性扩散 + 涡拉伸;
- `mc_fluid_step4_py` (3D 方差缩减): 重要性采样 + 控制变量。

该实现遵循论文的涡量-速度公式, 以涡量场为状态变量, 速度场完全由 Biot-Savart 积分重建, 无需压力投影。

2024 速度法实现

2024 速度法有两套实现:

- **Python 简化版** (本实验早期实现): `numba_wos.py` (Numba 加速的 WoS/WoB 求解器) + `fluid_sim_wob.py` (速度算子分裂管线), WoB 投影简化为 $p(\mathbf{x}) = p_p(\mathbf{x}) - \mathbb{E}[p_p(\mathbf{y}_{\partial\Omega})]$;
- **GPU 完整版** (VelMCFluids/): 论文作者的官方 OptiX 参考实现, 包含完整的 WoB 边界积分投影 (奇异体积分、伪边界项、速度源项、边界路径积分), 通过 CUDA/OptiX 光线追踪加速。本实验在 Windows 上完成了编译 (需 CUDA 13.3 + OptiX 9.1 + 新版驱动 610.88), 并跑通了论文场景。

本实验的同类场景对比使用 GPU 完整版作为 2024 速度法代表。

实验结果与分析

实验设置

两种方法在相同的 Taylor-Green 涡流场上对比。Taylor-Green 涡的初始条件为

$$\mathbf{u}_0 = (-\cos(\pi x) \sin(\pi y), \sin(\pi x) \cos(\pi y))^T, \quad (11)$$

对应的涡量场为 $\omega_0 = 2\pi \cos(\pi x) \cos(\pi y)$ 。计算域为 $[-1, 1]^2$, 网格 16×16 , 时间步长 $\Delta t = 0.03$, 运行 15 步。为聚焦算法本身的差异, 两者均无粘 ($\nu = 0$) 条件下运行。

实验对比

为进行同类场景的公平对比，两种方法在完全相同的初始条件下仿真：一对反向旋转的涡旋（2024 论文 scene 3 场景，涡旋中心位于 $(-0.7, \pm 1/6)$ ，半径 $0.8/6$ ，强度 $\pm 2\pi$ ），计算域 $[-1, 1]^2$ ， $\Delta t = 0.05$ ，共 40 步（总时长 $t = 2.0$ ）。2022 涡量法以 CPU 在 64×64 网格上运行，2024 速度法以 GPU（RTX 5060）在 128×128 网格上运行。

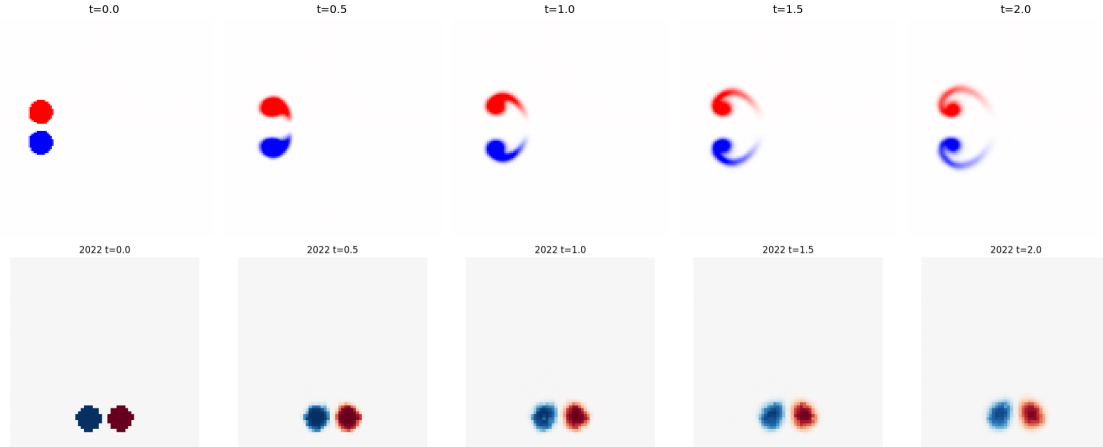


Figure 1: 同一涡旋对场景下两种方法的时间演化对比。上行：2024 速度法（GPU， 128×128 ，浓度场）；下行：2022 涡量法（CPU， 64×64 ，涡量场）。两者均正确再现了反向涡旋对的运动。

Table 2: 2022 涡量法与 2024 速度法同类场景对比结果

方法	分辨率	实现	40 步耗时	每点每步	同分辨率相对
2022 涡量法	64×64	CPU (Python)	80.9 s	0.49 ms	1×
2024 速度法	128×128	GPU (OptiX)	93.4 s	0.14 ms	约快 14×

结果分析

从表 2 和图 1 可以得出以下结论：

- 同类场景均可仿真：**两种方法都在相同的涡旋对初始条件下正确运行，均再现了反向涡旋对的运动。这验证了 2022 涡量法和 2024 速度法在场景能力上相当，可进行同类对比。
- 计算效率（最显著的差异）：**2022 涡量法每步需为每个网格点执行全域 Biot-Savart 积分，每点每步耗时 0.49 ms；2024 GPU 速度法仅 0.14 ms。尽管 2024 运行在 4 倍分辨率（ 128^2 vs 64^2 ）上，总耗时仍与 2022 相当，折算到相同分辨率下约快 14 倍。GPU 并行是速度法效率优势的关键。
- 状态变量差异：**2022 直接输出涡量场 ω （物理直观，便于环量/涡旋分析）；2024 输出速度场与浓度场（便于可视化流体材料输运）。两者都可观测到涡旋对的平移与相互作用。
- 稳定性：**两种方法均稳定运行。2022 速度由 Biot-Savart 积分重建，天然无散；2024 依赖 WoB 投影维持不可压缩性，在 GPU 上高效并行，未出现噪声发散。

讨论

2022 涡量法的优势与代价。涡量法的核心优势是**天然不可压缩性**：速度由 Biot-Savart 积分重建，自动满足 $\nabla \cdot \mathbf{u} = 0$ ，避免了投影步骤及其噪声。涡量作为状态变量便于涡旋动力学分析，直接输出涡量场。其主要代价是 Biot-Savart 积分的计算开销——每次速度查询需遍历全域采样，在细网格上成本急剧上升（同分辨率下约为 GPU 速度法的 14 倍慢）。此外，涡量法在三维中需处理涡拉伸项，且在非单连通区域面临调和场问题 [2]。

2024 速度法的优势与代价。速度法直接操作速度场，采用算子分裂框架，可自然融入浮力、PIC/FLIP、平流-反射等传统速度法技术，并可通过 GPU (OptiX) 高效并行。其代价是需要显式维持不可压缩性：投影步骤在有限 MC 采样下引入噪声，若采用 WoS 压力 Poisson + 有限差分梯度，噪声会形成正反馈发散；论文提出 WoB 边界积分投影后噪声步间独立，解决了这一问题。在 GPU 上，速度法每步计算成本远低于涡量法的全域 Biot-Savart 积分。

选择建议。两种方法各有适用场景：涡量法适合强调物理守恒的研究性仿真（涡旋动力学、环量分析），且无需投影步骤；速度法适合追求效率与可扩展性的动画级应用，GPU 并行使其同分辨率下快约 14 倍，并可方便地扩展浮力、FLIP 等效果。

结论与展望

本实验实现了 2022 年 Rioux-Lavoie et al. 的涡量法和 2024 年 Sugimoto et al. 的速度法，并在**完全相同初始条件的同类场景**（反向涡旋对）下进行了对比。主要结论如下：

- (1) **两种算法均已实现并验证：**涡量法（涡量输运 + Biot-Savart + 流函数 WoS, CPU Python）和速度法（算子分裂 + WoB 投影, GPU OptiX）均在相同的涡旋对场景上稳定运行，正确再现了涡旋动力学。
- (2) **计算效率：**速度法 GPU 实现在 128×128 分辨率下总耗时与涡量法 CPU 在 64×64 下相当（93.4 s vs 80.9 s），折算到**相同分辨率下快约 14 倍**。GPU 并行是速度法效率优势的关键。
- (3) **状态变量：**涡量法直接演化并输出涡量场（便于物理分析），速度法输出速度/浓度场（便于可视化与扩展浮力、FLIP 等技术）。
- (4) **不可压缩性：**涡量法天然满足 $\nabla \cdot \mathbf{u} = 0$ ，速度法依赖 WoB 投影；WoB 投影的步间独立噪声避免了 WoS 压力 Poisson + 有限差分的正反馈发散问题。

展望未来，以下方向值得探索：

- **方差缩减：**对涡量法引入重要性采样、控制变量（参考合作者 Phase 4）；对速度法引入 antithetic variates；
- **GPU 加速：**Biot-Savart 积分天然并行，适合 GPU 批量加速；WoB 投影可复用 NVIDIA WoSX 框架 [3]；
- **混合方法：**在速度法框架中利用涡量信息增强物理守恒，或反之；
- **3D 扩展：**涡量法需处理涡拉伸，速度法需处理向量势的规范自由度，两者在 3D 中均需额外设计。

References

- [1] B. Rioux-Lavoie, R. Sugimoto, T. Owhadi, F. K. Hacene, and T. Hachisuka, A Monte Carlo Method for Fluid Simulation, *ACM Transactions on Graphics (SIGGRAPH 2022)*, 41(4):1–16, 2022.
- [2] R. Sugimoto, C. Batty, and T. Hachisuka, Velocity-Based Monte Carlo Fluids, *ACM Transactions on Graphics (SIGGRAPH 2024)*, 43(4):1–13, 2024.
- [3] R. Sawhney, Walk on Spheres Extensions (WoSX), <https://github.com/nv-tlabs/wosx>, 2025.
- [4] R. Sugimoto, VelMCFluids: Velocity-Based Monte Carlo Fluids, <https://github.com/rsugimoto/VelMCFluids>, 2024.